

Inventory Management System (IMS)

Backend Algorithm & Architecture Documentation

1. Purpose of This Document

This document provides a **professional, production-grade algorithmic and architectural blueprint** for building a backend Inventory Management System (IMS). It is framework-agnostic but assumes a **RESTful API architecture** and is suitable for implementation using Node.js (Express/Nest), Django, Spring Boot, or similar backend stacks.

The goal is to ensure: - Scalability - Data integrity - Auditability - Maintainability - Clear separation of concerns

2. System Scope & Core Capabilities

2.1 Functional Scope

The backend system must support: - Product & category management - Inventory stock tracking - Supplier management - Purchase (stock-in) operations - Sales (stock-out) operations - Inventory adjustments - Low-stock alerts - Role-based access control (RBAC) - Transaction logs and audit trails - Reporting-ready data

2.2 Non-Functional Requirements

- ACID-compliant database operations
 - High concurrency safety
 - Secure authentication & authorization
 - Extensible architecture
 - Fault tolerance
-

3. High-Level Architecture

3.1 Layered Architecture

```
Client
↓
API Layer (Controllers)
↓
Service Layer (Business Logic)
↓
```

```
Domain / Models
  ↓
Repository / Data Access Layer
  ↓
Database
```

Each layer has **strict responsibility boundaries**.

4. Core Domain Models (Conceptual)

4.1 Product

```
Product
- id
- name
- sku (unique)
- category_id
- unit_of_measure
- cost_price
- selling_price
- reorder_level
- status (active/inactive)
- created_at
- updated_at
```

4.2 Inventory

```
Inventory
- id
- product_id
- quantity_on_hand
- quantity_reserved
- warehouse_location
- updated_at
```

4.3 Stock Transaction

```
StockTransaction
- id
- product_id
- type (IN | OUT | ADJUSTMENT | RETURN)
- quantity
```

- reference_id (sale_id / purchase_id)
- performed_by (user_id)
- created_at

4.4 Supplier

- Supplier
- id
 - name
 - contact_info
 - status
 - created_at

4.5 User & Roles

- User
- id
 - name
 - email
 - role (ADMIN | MANAGER | STAFF)
 - status

5. Core Algorithms & Workflows

5.1 Product Creation Algorithm

Objective

Safely create a new product with unique identifiers.

Algorithm

1. Validate input payload
2. Check SKU uniqueness
3. Create product record
4. Initialize inventory with quantity = 0
5. Commit transaction

Pseudocode

```
BEGIN TRANSACTION
IF sku EXISTS → THROW error
CREATE product
CREATE inventory(product_id, quantity = 0)
COMMIT
```

5.2 Stock-In (Purchase) Algorithm

Objective

Increase inventory accurately while maintaining audit history.

Algorithm

1. Validate product and supplier
2. Validate quantity > 0
3. Lock inventory row (pessimistic or optimistic lock)
4. Increase quantity_on_hand
5. Record stock transaction (IN)
6. Commit transaction

Pseudocode

```
BEGIN TRANSACTION
LOCK inventory WHERE product_id
inventory.qty += input.qty
INSERT stock_transaction(type=IN)
COMMIT
```

5.3 Stock-Out (Sales) Algorithm

Objective

Prevent negative stock and ensure consistency.

Algorithm

1. Validate product existence
2. Check available stock >= requested quantity
3. Lock inventory record

4. Decrease quantity_on_hand
5. Create stock transaction (OUT)
6. Commit transaction

Failure Conditions

- Insufficient stock
- Concurrent update conflict

Pseudocode

```
BEGIN TRANSACTION
LOCK inventory
IF inventory.qty < requested → ABORT
inventory.qty -= requested
INSERT stock_transaction(type=OUT)
COMMIT
```

5.4 Inventory Adjustment Algorithm

Objective

Handle corrections, damages, audits.

Algorithm

1. Validate authorization (Manager/Admin only)
2. Capture reason for adjustment
3. Lock inventory
4. Apply positive or negative adjustment
5. Log transaction (ADJUSTMENT)

5.5 Low Stock Detection Algorithm

Objective

Trigger alerts when stock falls below reorder level.

Algorithm

1. After every stock mutation
2. Compare quantity_on_hand vs reorder_level
3. If below threshold → enqueue alert

Pseudocode

```
IF inventory.qty <= product.reorder_level  
  TRIGGER notification
```

6. Concurrency Control Strategy

6.1 Recommended Approach

- **Database-level transactions**
- Row-level locking for inventory table

6.2 Why This Matters

Prevents: - Overselling - Race conditions - Phantom reads

7. Authorization & Security Algorithm

Role Permissions Matrix

Action	Admin	Manager	Staff
Create Product	Yes	Yes	No
Stock Adjustment	Yes	Yes	No
Stock In/Out	Yes	Yes	Yes
View Reports	Yes	Yes	No

Authorization Flow

1. Authenticate user (JWT / Session)
 2. Extract role
 3. Validate permission before service execution
-

8. Error Handling Strategy

Principles

- Fail fast
- Clear error codes
- No partial writes

Common Errors

- SKU_DUPLICATE
 - INSUFFICIENT_STOCK
 - UNAUTHORIZED_ACTION
 - CONCURRENT_MODIFICATION
-

9. Audit & Logging Algorithm

What to Log

- Every stock mutation
- User performing the action
- Timestamp
- Before/after quantity

Storage

- Dedicated `stock_transactions` table
 - Immutable records
-

10. Reporting Readiness

Design ensures: - Daily stock movement reports - Product performance analysis - Supplier delivery history

No heavy joins during runtime; reports can be offloaded to analytics services.

11. Scalability Considerations

- Stateless API design
 - Horizontal scaling
 - Read replicas for reporting
 - Event-driven extensions (Kafka / RabbitMQ)
-

12. Extension Points

Future modules can include: - Multi-warehouse support - Batch & expiry tracking - Barcode scanning - Accounting system integration

13. Summary

This algorithmic documentation defines a **robust, enterprise-grade backend foundation** for an Inventory Management System. Following this structure ensures correctness, scalability, and long-term maintainability.

If required, this document can be converted into: - Database schema (SQL) - REST API contract (OpenAPI) - Modular backend codebase (Node.js / Django / Spring)